

LIBERTAS

Curso: Sistemas de Informação

Disciplina: Estrutura de Dados II

Professor: MSc. Jonathan Henrique Jeremias Souza

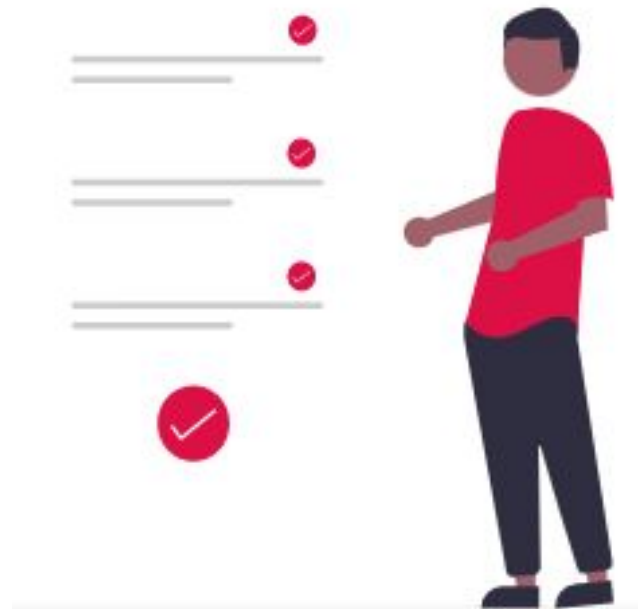
E-mail: jonathansouza@libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

AGENDA

- Introdução ao Hashing e Funções Hash
- Funções Hash: exemplos e estratégias
- Resolução de Colisões: Open Hashing e Closed Hashing



INTRODUÇÃO AO HASHING

O que é hashing e por que é eficiente

Hashing é um método para armazenar e recuperar registros de um banco de dados. Permite inserir, excluir e buscar registros a partir de um valor de chave.

Quando implementado corretamente, essas operações podem ser realizadas em tempo constante $O(1)$. Um sistema hash bem ajustado examina apenas um ou dois registros por operação.

Isso é muito melhor que o custo médio $O(\log n)$ de uma busca binária em um array ordenado, ou o custo $O(\log n)$ de uma operação em uma árvore de busca binária.

INTRODUÇÃO AO HASHING

Hashing é simples na ideia, mas difícil de implementar corretamente

Embora o hashing seja baseado em uma ideia muito simples, é surpreendentemente difícil de implementar de forma correta.

Os desenvolvedores precisam prestar atenção cuidadosa a todos os detalhes envolvidos na implementação de um sistema hash.

Nesta aula veremos os fundamentos necessários para construir uma implementação correta e eficiente.

INTRODUÇÃO AO HASHING

A estrutura de uma tabela hash: HT, função h e slots

Um sistema hash armazena registros em um array chamado tabela hash, que chamaremos de HT.

O hashing funciona realizando um cálculo sobre uma chave de busca K para identificar a posição em HT que contém o registro com chave K .

A função que realiza esse cálculo é chamada de função hash, denotada pela letra h .

INTRODUÇÃO AO HASHING

A estrutura de uma tabela hash: HT, função h e slots

Uma posição na tabela hash é também conhecida como slot.

O número de slots em HT será denotado por M , com slots numerados de 0 a $M-1$.

O objetivo é que, para qualquer chave K e função hash h , o valor $i = h(K)$ seja um slot tal que $0 \leq i < M$, e o registro armazenado em $HT[i]$ tenha chave igual a K .

INTRODUÇÃO AO HASHING

Quando usar hashing, e quando não usar

Hashing não é adequado para aplicações onde múltiplos registros com o mesmo valor de chave são permitidos.

Hashing não é um bom método para responder buscas por intervalo.

Não conseguimos encontrar facilmente todos os registros cujas chaves estejam em um determinado intervalo, nem o registro com valor mínimo ou máximo, nem visitar os registros em ordem.

INTRODUÇÃO AO HASHING

Quando usar hashing, e quando não usar

Hashing é mais apropriado para responder a pergunta:

"Qual registro, se algum, tem o valor de chave K?"

Para aplicações onde toda busca é feita por consultas de correspondência exata (exact-match), hashing é o método de busca preferido.

INTRODUÇÃO AO HASHING

Exemplo simples e o problema do espaço

Como exemplo simples, considere armazenar n registros, cada um com uma chave única no intervalo de 0 a $n-1$.

Um registro com chave k pode ser armazenado em $HT[k]$, e a função hash é $h(k) = k$.

Na maioria das aplicações, há muito mais valores no intervalo de chaves do que slots na tabela hash.

INTRODUÇÃO AO HASHING

Exemplo simples e o problema do espaço

Por exemplo, se a chave pode assumir qualquer valor de 0 a 65.535 e esperamos armazenar cerca de 1.000 registros, é impraticável usar uma tabela com 65.536 slots.

Precisamos então de uma função hash que permita armazenar os registros em uma tabela muito menor.

Como o intervalo de chaves é maior que o tamanho da tabela, ao menos alguns slots receberão mapeamentos de múltiplos valores de chave.

INTRODUÇÃO AO HASHING

Colisão: quando duas chaves mapeiam para o mesmo slot

Dados uma função hash h e duas chaves k_1 e k_2 , se $h(k_1) = \beta = h(k_2)$, onde β é um slot na tabela, dizemos que k_1 e k_2 têm uma colisão no slot β sob a função hash h .

Encontrar um registro com chave K em um banco de dados organizado por hashing segue um procedimento de dois passos:

- (1) calcular a posição $h(K)$ na tabela;
- (2) a partir do slot $h(K)$, localizar o registro contendo a chave K usando, se necessário, uma política de resolução de colisão.

INTRODUÇÃO AO HASHING

Colisão: quando duas chaves mapeiam para o mesmo slot

A política de resolução de colisão é um dos aspectos mais críticos do projeto de um sistema hash eficiente.

FUNÇÕES HASH: EXEMPLOS E ESTRATÉGIAS

Colisões são quase inevitáveis, a analogia do aniversário

Colisões ocorrem quando dois registros fazem hash para o mesmo slot.

Se tivermos cuidado, ou sorte, ao escolher uma função hash, o número real de colisões será pequeno.

Infelizmente, mesmo nas melhores circunstâncias, colisões são quase inevitáveis.

FUNÇÕES HASH: EXEMPLOS E ESTRATÉGIAS

Colisões são quase inevitáveis, a analogia do aniversário

Para ilustrar: em uma sala com 23 estudantes, a probabilidade de dois compartilharem o mesmo aniversário é de aproximadamente 50%, mesmo com 365 dias disponíveis. O mapeamento de estudantes para dias é similar ao mapeamento de registros para slots.

Para ser prático, um banco de dados organizado por hashing deve usar uma tabela que não desperdice espaço.

Para equilibrar eficiência de tempo e espaço, a tabela hash deve estar aproximadamente na metade da capacidade.

FUNÇÕES HASH: EXEMPLOS E ESTRATÉGIAS

O que faz uma função hash ser boa

Gostaríamos de escolher uma função hash que mapeie chaves para slots de forma que cada slot tenha igual probabilidade de ser preenchido para o conjunto real de chaves sendo usado.

Infelizmente, normalmente não temos controle sobre a distribuição dos valores de chave para os registros reais.

Dados reais raramente são uniformemente distribuídos, podem seguir distribuições de Zipf, ser arredondados para valores específicos ou ter letras iniciais mal distribuídas.

FUNÇÕES HASH: EXEMPLOS E ESTRATÉGIAS

O que faz uma função hash ser boa

Ao projetar funções hash, enfrentamos duas situações:

- (1) não sabemos nada sobre a distribuição das chaves, devemos escolher uma função que distribua uniformemente o intervalo de chaves;
- (2) conhecemos algo sobre a distribuição, devemos usar uma função dependente dessa distribuição para evitar clustering.

FUNÇÕES HASH: EXEMPLOS E ESTRATÉGIAS

Função simples de módulo (mod)

Considere a seguinte função hash para mapear inteiros em uma tabela de M slots. Aqui "%" é o símbolo da operação módulo.

O valor retornado por essa função depende exclusivamente dos bits menos significativos da chave. Como esses bits tendem a ser mal distribuídos, por exemplo, muitas chaves podem ser números pares, fazendo com que o bit de menor ordem seja sempre zero, o resultado também será mal distribuído.

Este exemplo mostra que o tamanho da tabela M pode ter um grande efeito no desempenho do sistema hash, pois M é tipicamente usado como módulo para garantir que a função produza um

FUNÇÕES HASH: EXEMPLOS E ESTRATÉGIAS

Função simples de módulo (mod)

Este exemplo mostra que o tamanho da tabela M pode ter um grande efeito no desempenho do sistema hash, pois M é tipicamente usado como módulo para garantir que a função produza um número no intervalo de 0 a M-1.

```
// Função hash simples de módulo - Java
int h(int x) {
    return x % 16;
}
```

MUITO OBRIGADO!

Ficou com alguma
dúvida?



MSc. Jonathan Henrique Jeremias Souza
jonathansouza@libertas.edu.br
www.libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

Referências Bibliográficas

- SHAFFER, Clifford A. Introduction to Hashing. OpenDSA Data Structures and Algorithms. Disponível em: <https://opendsa-server.cs.vt.edu/OpenDSA/Books/Everything/html/HashIntro.html>. Acesso em: 01 jun. 2026.
- SHAFFER, Clifford A. Hash Function Principles. OpenDSA Data Structures and Algorithms. Disponível em: <https://opendsa-server.cs.vt.edu/OpenDSA/Books/Everything/html/HashFunc.html>. Acesso em: 01 jun. 2026.
- SHAFFER, Clifford A. Sample Hash Functions. OpenDSA Data Structures and Algorithms. Disponível em: <https://opendsa-server.cs.vt.edu/OpenDSA/Books/Everything/html/HashFuncExamp.html>. Acesso em: 01 jun. 2026.
- SHAFFER, Clifford A. Open Hashing. OpenDSA Data Structures and Algorithms. Disponível em: <https://opendsa-server.cs.vt.edu/OpenDSA/Books/Everything/html/OpenHash.html>. Acesso em: 01 jun. 2026.
- SHAFFER, Clifford A. Collision Resolution. OpenDSA Data Structures and Algorithms. Disponível em: <https://opendsa-server.cs.vt.edu/OpenDSA/Books/Everything/html/HashCSimple.html>. Acesso em: 01 jun. 2026.

TIPOS DE DADOS EM JAVASCRIPT

Operações matemáticas são seguras em JavaScript

- ...
- ...
- ...